PREDICTIVE SWARM ENGINE

Adversarial Multi-Agent Debate for Causal-Chain Forecasting

STUDENT INFO

Submitted By: Ichhit Karwa

Category: Agent Swarm

PROJECT TECH STACK

Core Logic & Runtime: Python 3.14 / SQLite / NumPy / NetworkX (Centrality)

AI Orchestration: Google Gemini API (3.1 Flash-Lite & 3.5 Flash)

Interface & Visuals: Streamlit / Matplotlib / 2D Force-Graph (D3 / Force-Graph JS)

Compilation: pypdf / ReportLab (Dynamic Slide Engine)

The Problem: Cognitive Limitations of Single LLMs

1. Confirmation Bias & Monologue

Single LLM agents naturally lock onto initial hypotheses early in their context window. They lack the cognitive friction needed to self-critique, leading to narrow, biased forecasts.

2. Persona Contamination & Dilution

Strategic forecasts (e.g. market shifts, regulatory impact) require specialized perspectives (Legal, Finance, Risk). A single agent prompt cannot hold conflicting expert viewpoints without context mix.

3. Causal Chain Hallucinations

Forecasting is about tracking causal ripple effects (A causes B, which triggers C). Without grounding models in structured knowledge, single LLMs fail to trace actual dependencies and hallucinate connections.

The Solution: Grounded Cooperative-Adversarial Swarms

1. Causal Graph Grounding (GraphRAG)

Ingests raw research PDFs and parses them into an SQLite-backed semantic network. Implements stem-based fuzzy Jaccard entity resolution to deduplicate and establish a clean, canonical graph.

2. Director Agent Swarm Spawning

Evaluates the user's forecast query to outline the required domain expertise. Automatically generates and spawns 5 custom specialist personas with isolated system prompts.

3. Dialectical Multi-Round Debate

Triggers a structured, 2-round peer-critique debate. Specialists retrieve query-relevant subgraphs, outline causal paths, and actively challenge peer arguments to expose weaknesses.

4. Executive Synthesis & Visualization

A high-reasoning engine reviews the debate logs, resolves conflicts, and compiles a synthesis report. A visualization agent dynamically outputs custom, sandboxed matplotlib charts.

System Architecture: End-to-End Pipeline

1. Semantic Ingestion & Chunking

Splits source documents into overlapping blocks. Extracted chunks are analyzed by an Architect Agent to generate a custom spaced ontology schema.

2. Graph Extraction & Resolution

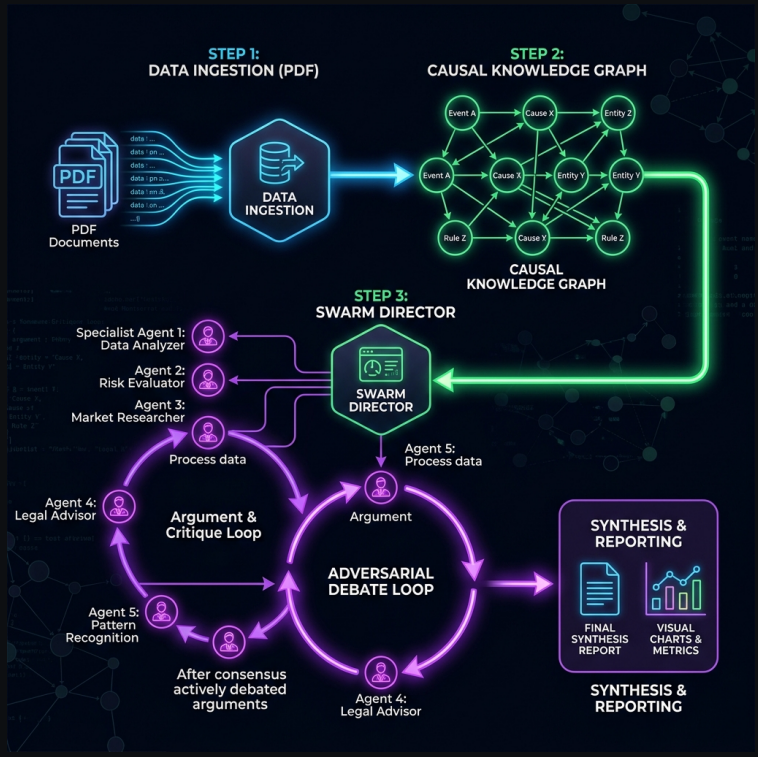
Uses sliding-window LLM passes to extract entity-relationship tuples. Identical entities are merged using stem-based Jaccard similarity at a 0.65 threshold.

3. Centrality-Guided Retrieval

Computes node degree centrality to mathematically highlight key causal risks. Matches user queries to localized subgraphs using text-embeddings.

4. Swarm Debate & Synthesis Loop

Orchestrates 5 spawned experts in a 2-round critique loop, utilizing external search tools and generating visual charts for final synthesis.



Dialectical Reasoning: Swarm vs. Single-Agent

Dimension	Single-Agent (Traditional RAG)	Multi-Agent Collaborative Swarm (Ours)
Orchestration	Monologue: Linear output from a single context window. No internal validation or critique of key assumptions.	Dialectical: 5 domain specialists argumentatively challenge and critique forecasts over 2 structured debate rounds.
Persona Depth	Generic: A single system prompt attempts to balance conflicting priorities, watering down analytical depth.	Custom Experts: Spawns targeted specialists (e.g. Legal, Finance, Supply Chain) with completely isolated, dedicated prompts.
Error Correction	Vulnerable: Highly susceptible to confirmation bias, path dependency, and hallucinated causal relationships.	Active Auditing: Mutual criticism forces agents to justify claims, check source quotes, and expose logical gaps.
Context Efficiency	Inefficient: Injects the entire retrieved text context, causing cognitive dilution and distraction.	Modular: Retrieves query-relevant subgraphs tailored specifically to each specialist's analytical lens.

AI Integration: Tiered LLM Routing & Prompts

Tiered Model Routing Architecture

• **Gemini 3.1 Flash-Lite (High-Speed Swarm Debate)**
Orchestrates extraction and the debate rounds. Handles $O(N)$ operations. Spawning 5 specialists and executing 10 turn debates runs concurrently, respecting API rate limits (15 RPM / 90k TPM) while keeping execution speed high.

• **Gemini 3.5 Flash (High-Reasoning Synthesis)**
Synthesizes the final forecast. Evaluates core points of agreement/conflict, calculates statistical confidence scores, and determines visual visualization requirements.

Graph-Augmented Retrieval Routing

Uses semantic text embeddings to match the trigger query to localized subgraphs, passing degree centrality metrics directly to specialists to ground their debate.

SPECIALIST SPAWNING CONFIGURATION

```
# Spawning Spaced Personas JSON:
[
  {
    "name": "Macroeconomic Risk Analyst",
    "focus": "Liquidity & supply chains",
    "system_prompt": "You are a Macro Analyst.
Focus on corporate debt, interest rates,
merchant liquidity, and cash flow velocity..."
  },
  {
    "name": "Regulatory Compliance Counsel",
    "focus": "RBI circulars & licensing limits",
    "system_prompt": "You are a Regulatory Counsel.
Focus on banking mandates, KYC guidelines,
penal audit risks, and compliance..."
  }
]
```

Autonomous Data Visualization Subsystem

Visual Forecast Code Generation & Sandbox

• **Matplotlib Code Generation:** The *DataVisualizerAgent* reviews the debate history, parses numerical predictions, and generates clean python matplotlib code.

• **Headless Server Guard:** Automatically injects matplotlib.use('Agg') to disable GUI rendering loops, preventing system-level crashes on Streamlit Cloud.

• **Cyberpunk Styling Enforcement:** Limits drawing colors to neon palettes (cyan, purple, pink) with solid dark backgrounds matching the main app's glassmorphism style.

• **Restricted Namespace Sandboxing:** Runs execution under a scope-restricted exec() block containing only numpy and pyplot modules, blocking shell injections.

SANDBOXED PYTHON EXECUTION LOGIC

```
def execute_visualization_code(code_str):
    # Restrict execution namespace
    local_scope = {
        'np': numpy,
        'plt': pyplot,
        'output_path': 'scratch/temp_chart.png'
    }
    try:
        # Run inside empty globals sandbox
        exec(code_str, {}, local_scope)
        return os.path.exists(
            'scratch/temp_chart.png'
        )
    except Exception as e:
        logger.error(f"Execution failed: {e}")
        return False
```

Interactive Streamlit Web Interface

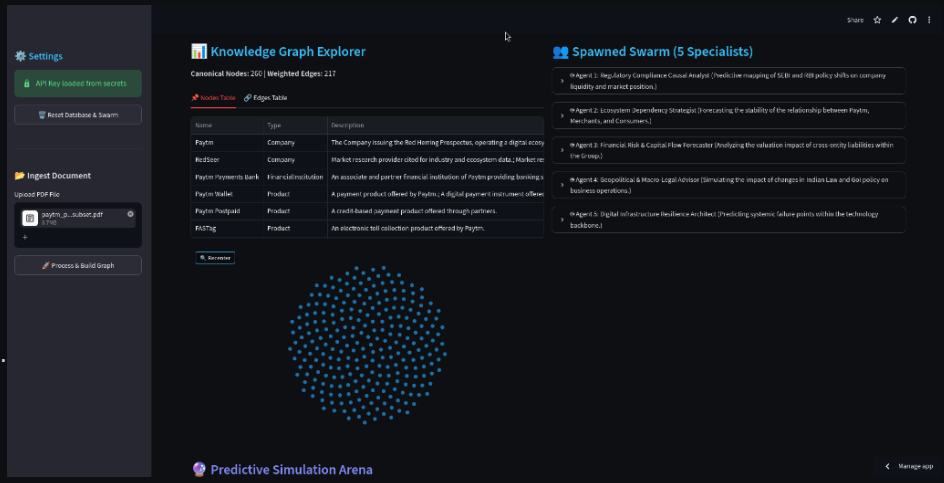
High-Fidelity Simulation Dashboard

• **Causal Knowledge Graph Explorer:** Renders interactive 2D Force-Directed Graphs using D3/Force-Graph JS. Displays node tooltips with descriptions and supports drag/zoom actions.

• **Swarm Persona Inspector:** Displays detailed cards for each dynamically generated specialist, showing their system prompts and target focus areas.

• **Simulation Arena:** Features a trigger interface for 'What-If' scenarios. Displays the live 2-round debate log side-by-side with the final synthesis report.

• **Visual Chart & Code Inspector:** Automatically displays matplotlib outputs and provides a dropdown to review generated code for auditing.



Thesis Case Study: Paytm IPO Backtest

Redacted Historical Evaluation

We ingested Paytm's 2021 pre-IPO Prospectus, completely redacting all post-2021 news, and triggered the scenario: *'What if the RBI bans Paytm Payments Bank (PPBL) from onboarding new customers or accepting deposits?'*

The Swarm's Predictive Accuracy:

The 5-agent swarm successfully predicted that Paytm would lose self-clearing status and pivot to a low-margin B2B utility rail, partnering with HDFC, Axis, and SBI (exactly matching March 2024 events).

Step	Swarm-Predicted Causal Chain Path
1	**Regulatory Decoupling:** RBI bans PPBL, halting self-clearing.
2	**Network Churn:** Merchants abandon Paytm QR nodes.
3	**Moat Collapse:** Loss of deposits stops the lending engine.
4	**M&A; Lock:** Compliance liabilities freeze corporate buyouts.
5	**Distressed Pivot:** Forced transition to B2B tech licensing.
6	**Discounted Exit:** Listing acts as low-multiple exit.

Probability & Sensitivity Analysis

- **80% Base Case:** Forced structural split and pivot to a commoditized B2B 'Utility-as-a-Service' bank rail.
- **10% Downside:** Runaway merchant churn causing insolvency.
- **10% Upside:** Regulatory pardon with a valuation haircut.

Project Team & Future Vision

PROJECT AUTHOR

Ichhit Karwa
Lead AI Architect & Full-Stack Developer

Designed the entity resolution engine, graph-retrieval routing logic, and the multi-agent dialectical debate loop.

ARCHITECTURAL VISION

To transition Large Language Models from simple text summarizers and fact search engines into active, grounded simulators for strategic causal-chain forecasting.

FUTURE RESEARCH DIRECTIONS

- **Scale Swarms to 100+ Personas**
  Implement hierarchical sub-swarms where groups debate sub-components (e.g., specific legal sub-clausals or microeconomic factors) before feeding into the Director.

- **High-Dimensional Entity Clustering**
  Utilize vector embeddings to automatically merge nodes and entities that represent the same canonical concept, improving graph density and search precision.

- **Real-Time Knowledge Graph Synced Streams**
  Connect the ingestion pipeline to live RSS feeds, financial wires, and social APIs to continuously update the graph while retaining historical checkpoints for backtesting.